Part C – Linux Environment Setup

The objective of this task is to understand the Linux environment and practice basic terminal commands used for file management, directory navigation, file operations, permissions, and software installation.

The task also involves setting up and verifying a Linux-based VLSI development environment.

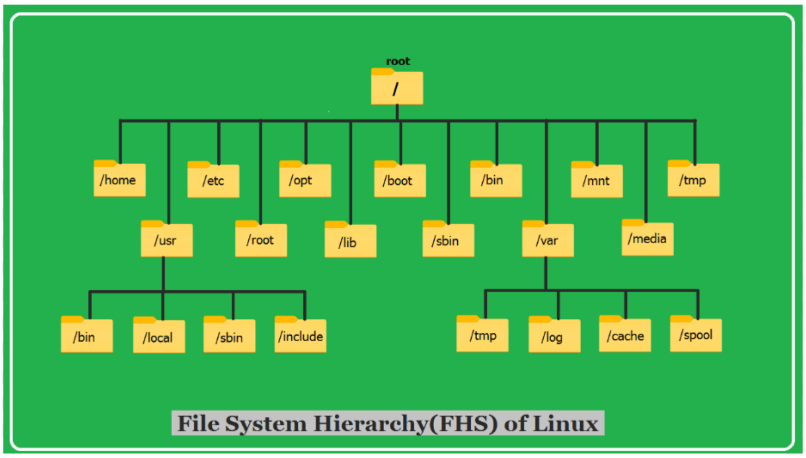
Topics Covered

- Linux File System
- Terminal Commands
- Directory Navigation
- File Operations
- Package Installation
- VLSI Development Environment Setup

Linux File System:-

The Linux file system is organized in a single, unified tree structure governed by the Filesystem Hierarchy Standard (FHS). Unlike Windows, which uses separate drive letters (like `C:` or `D:`), everything in Linux branches out from a single **root directory**, represented by a forward slash (`/`). [1, 2]

File System Hierarchy of Linux:-



File System Hierarchy(FHS) of Linux

Directory **Purpose**

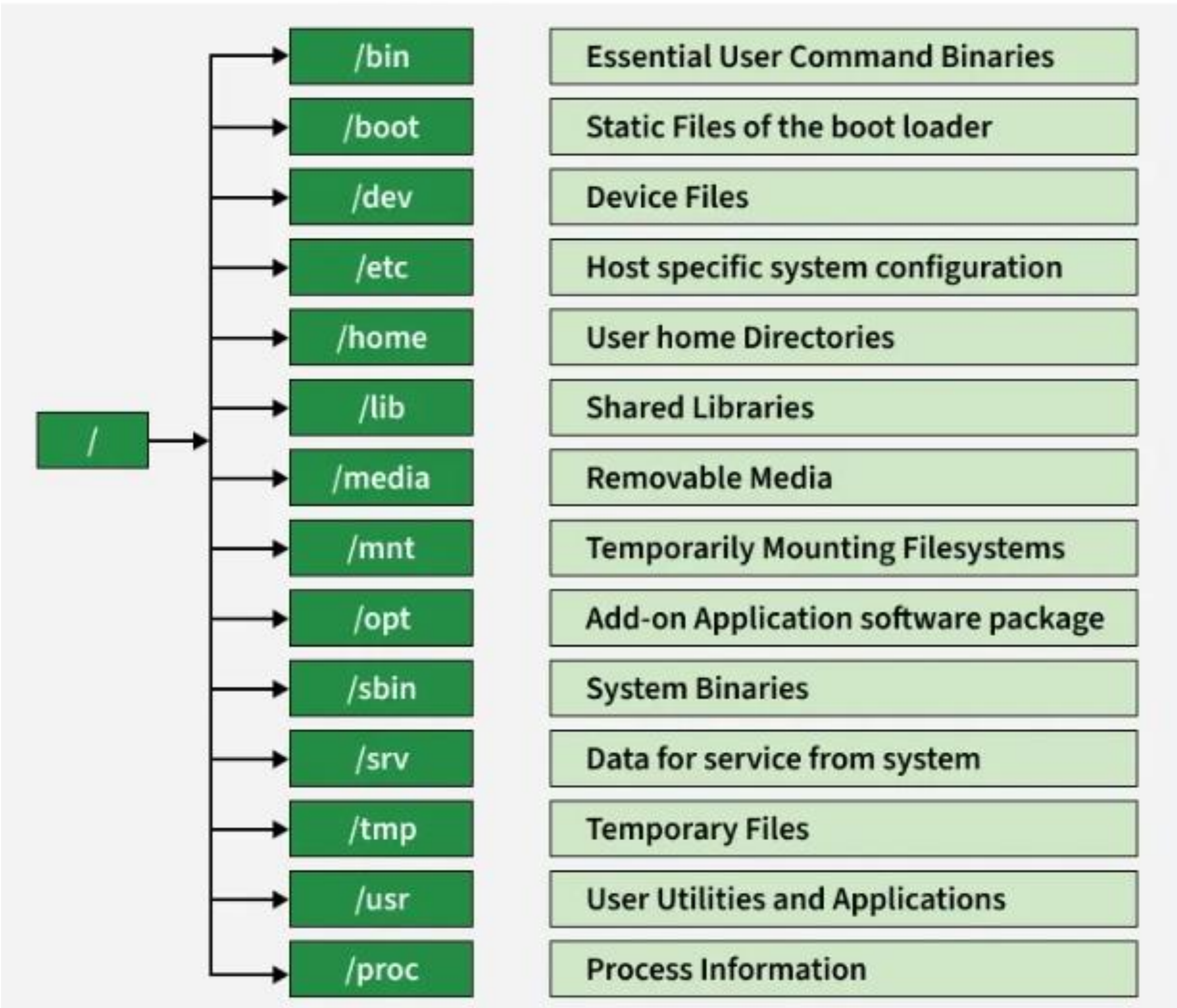
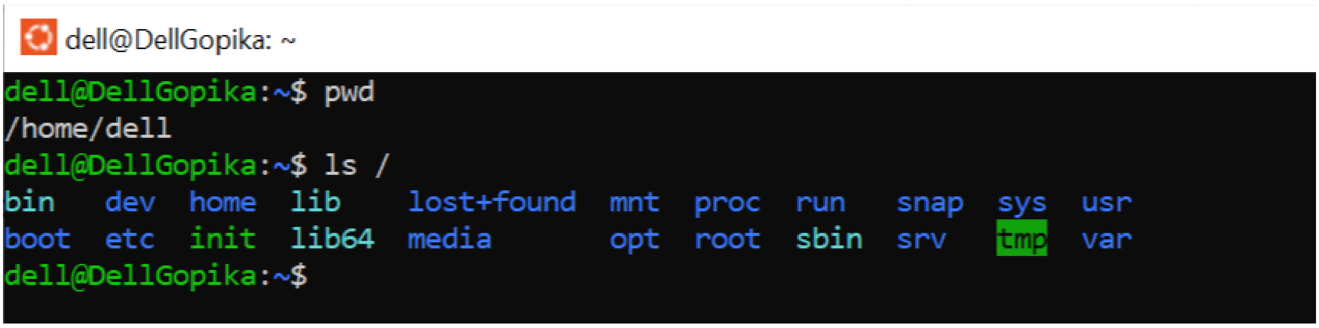


Figure 1: Linux root directory and its contents



The **`pwd`** command was used to identify the current working directory. The **`ls /`** command was then used to display the directories present in the Linux root directory.

2. Basic Linux Terminal Commands

Command	Function	Example
`pwd`	Displays the current working directory	`pwd`
`ls`	Lists files and directories	`ls`
`cd`	Changes the current directory	`cd vlsi`
`mkdir`	Creates a new directory	`mkdir test`
`cp`	Copies files or directories	`cp a.txt b.txt`
`mv`	Moves or renames files	`mv a.txt b.txt`
`rm`	Removes files	`rm a.txt`
`cat`	Displays file contents	`cat hello.sh`
`nano`	Creates/edits text files	`nano hello.sh`
`chmod`	Changes file permissions	`chmod +x hello.sh`

Linux terminal commands are essential tools that allow you to interact directly with the operating system using text-based instructions.

Mastering these fundamental commands gives you precise control over your system, enabling you to manage files, monitor system performance, and automate tasks efficiently.

3. Directory Navigation:-

Step 1: Check current directory:

The `pwd` command was used to display the current working directory.

Step 2: List files:

The `ls` command was used to display the files and directories present in the current directory.

Step 3: Create a directory:

The `mkdir` command was used to create a new directory named `vlsi`.

Step 4: Enter the directory:

The `cd` command was used to navigate into the newly created `vlsi` directory.

dell@DellGopika: ~/vlsi_1

```
dell@DellGopika:~$ pwd
/home/dell
dell@DellGopika:~$ ls /
bin   dev  home  lib    lost+found  mnt  proc  run   snap  sys  usr
boot  etc  init  lib64  media       opt  root  sbin  srv   tmp  var
dell@DellGopika:~$ mkdir vlsi_1
dell@DellGopika:~$ cd vlsi_1
dell@DellGopika:~/vlsi_1$ pwd
/home/dell/vlsi_1
dell@DellGopika:~/vlsi_1$
```

4. Creating and Editing a File:-

```
dell@DellGopika:~/vlsi_1$ pwd
/home/dell/vlsi_1
dell@DellGopika:~/vlsi_1$ nano hello.sh
dell@DellGopika:~/vlsi_1$
```

The **nano** text editor was used to create and edit a shell script named **hello.sh**.

Figure: Creating **hello.sh** using the nano editor

dell@DellGopika: ~/vlsi_1

```
  GNU nano 8.7.1
#!/bin/bash
echo "Hello VLSI"
```

5. Displaying File Contents:-

The **cat** command was used to display the contents of the `hello.sh` file.

dell@DellGopika: ~/vlsi_1

```
dell@DellGopika:~$ pwd
/home/dell
dell@DellGopika:~$ ls /
bin   dev  home  lib    lost+found  mnt  proc  run   snap  sys  usr
boot  etc  init  lib64  media       opt  root  sbin  srv   tmp  var
dell@DellGopika:~$ mkdir vlsi_1
dell@DellGopika:~$ cd vlsi_1
dell@DellGopika:~/vlsi_1$ pwd
/home/dell/vlsi_1
dell@DellGopika:~/vlsi_1$ nano hello.sh
dell@DellGopika:~/vlsi_1$ cat hello.sh
#!/bin/bash
echo "Hello VLSI"

dell@DellGopika:~/vlsi_1$
```

6. Changing File Permissions:-

The **`chmod`** command was used to provide execute permission to the shell script.

```
dell@DellGopika:~/vlsi_1$ nano hello.sh
dell@DellGopika:~/vlsi_1$ cat hello.sh
#!/bin/bash
echo "Hello VLSI"

dell@DellGopika:~/vlsi_1$ chmod +x hello.sh
dell@DellGopika:~/vlsi_1$ ./hello.sh
Hello VLSI
dell@DellGopika:~/vlsi_1$ 
```

Figure: Executing the shell script after changing its permissions

7.Copying a File:-

The **`cp`** command was used to create a copy of **`hello.sh`** named **`hello_copy.sh`**.

```
dell@DellGopika:~$ chmod +x hello.sh
dell@DellGopika:~$ ./hello.sh
HEllo VLSI
dell@DellGopika:~$ cp hello.sh hello_copy.sh
dell@DellGopika:~$ ls
hello.sh  hello_copy.sh  partc  vlsi  vlsi_1  vlsi_2
dell@DellGopika:~$
```

8. Moving/Renaming a File:-

The **`mv`** command was used to rename **`hello_copy.sh`** to **`renamed.sh`**.

```
dell@DellGopika:~$ cp hello.sh hello_copy.sh
dell@DellGopika:~$ ls
hello.sh  hello_copy.sh  partc  vlsi  vlsi_1  vlsi_2
dell@DellGopika:~$ mv hello_copy.sh renamed.sh
dell@DellGopika:~$ ls
hello.sh  partc  renamed.sh  vlsi  vlsi_1  vlsi_2
dell@DellGopika:~$
```

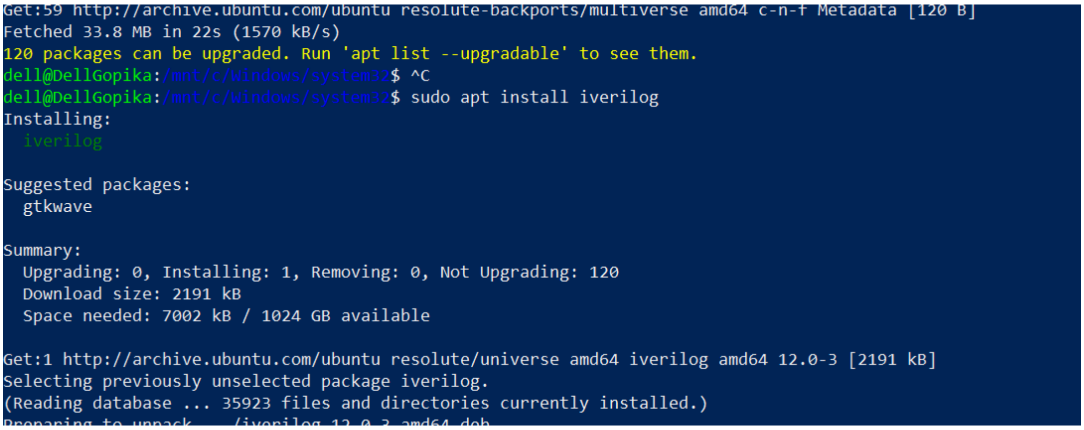
9.Removing a File:-

The **`rm`** command was used to remove the unwanted file **`renamed.sh`**.

```
dell@DellGopika:~$ mv hello_copy.sh renamed.sh
dell@DellGopika:~$ ls
hello.sh  partc  renamed.sh  vlsi  vlsi_1  vlsi_2
dell@DellGopika:~$ rm renamed.sh
dell@DellGopika:~$ ls
hello.sh  partc  vlsi  vlsi_1  vlsi_2
dell@DellGopika:~$
```

VLSI Development Environment:-

6.1 Icarus Verilog Installation and Verification



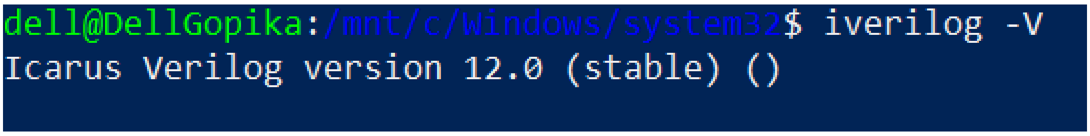
```
Get:59 http://archive.ubuntu.com/ubuntu resolute-backports/multiverse amd64 c-n-f Metadata [120 B]
Fetched 33.8 MB in 22s (1570 kB/s)
120 packages can be upgraded. Run 'apt list --upgradable' to see them.
dell@DellGopika:/mnt/c/Windows/system32$ ^C
dell@DellGopika:/mnt/c/Windows/system32$ sudo apt install iverilog
Installing:
  iverilog

Suggested packages:
  gtkwave

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 120
  Download size: 2191 kB
  Space needed: 7002 kB / 1024 GB available

Get:1 http://archive.ubuntu.com/ubuntu resolute/universe amd64 iverilog amd64 12.0-3 [2191 kB]
Selecting previously unselected package iverilog.
(Reading database ... 35923 files and directories currently installed.)
```

Icarus Verilog was installed in the Ubuntu environment using the APT package manager. The installation was verified using the `iverilog -V` command, which displayed Icarus Verilog version 12.0

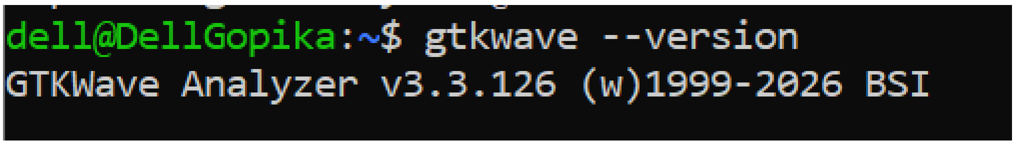


```
dell@DellGopika:/mnt/c/Windows/system32$ iverilog -V
Icarus Verilog version 12.0 (stable) ()
```

6.2 GTKWave Installation and Verification:-

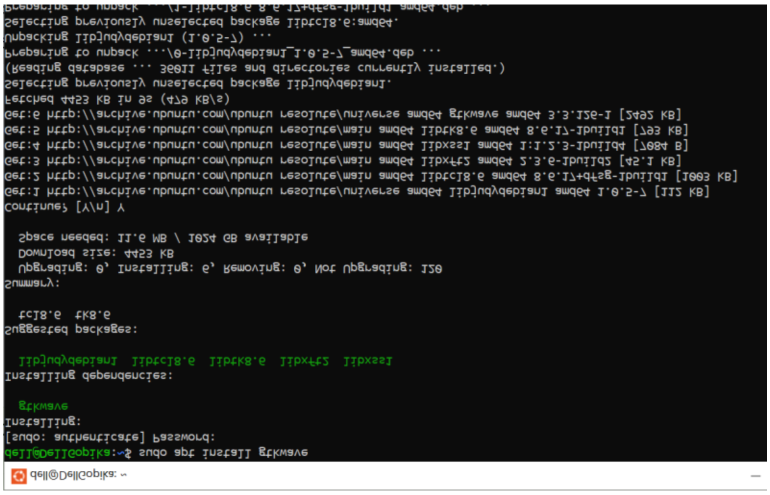
GTKWave was installed in the Ubuntu environment and verified using the `gtkwave --version` command.

The installed version was GTKWave Analyzer v3.3.126.



```
dell@DellGopika:~$ gtkwave --version
GTKWave Analyzer v3.3.126 (w)1999-2026 BSI
```

Figure 7.2: Verification of GTKWave installation



6.3 Yosys Installation and Verification:-

Yosys was installed in the Ubuntu environment using the APT package manager. The installation was verified using the `yosys -V` command, which displayed Yosys version 0.52

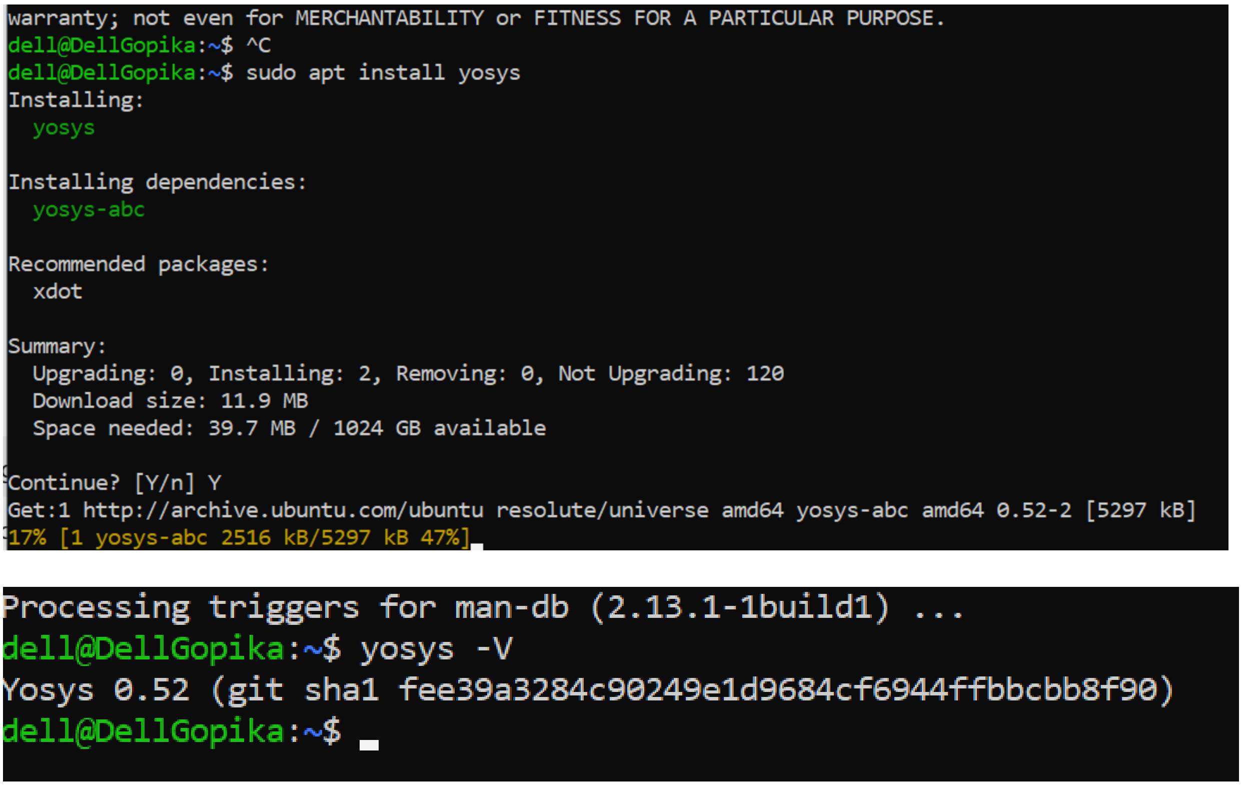


Figure 7.3: Installation and verification of Yosys

Activity: configure the VLSI development environment.

A simple AND gate was implemented using Verilog HDL and tested using a Verilog testbench. Icarus Verilog was used to compile and simulate the design.

A VCD waveform file was generated during simulation and viewed using GTKWave. The simulation produced the expected AND-gate outputs for all four input combinations.

```
dell@DellGopika:~$ mkdir ~/vlsi_test
dell@DellGopika:~$ cd ~/vlsi_test
dell@DellGopika:~/vlsi_test$ pwd
/home/dell/vlsi_test
dell@DellGopika:~/vlsi_test$ ^C
dell@DellGopika:~/vlsi_test$ nano and_gate.v
dell@DellGopika:~/vlsi_test$ ls
and_gate.v
dell@DellGopika:~/vlsi_test$ cat and_gate.v
module and_gate(input a, input b,output y);
assign y = a&b;
endmodule

dell@DellGopika:~/vlsi_test$
```

dell@DellGopika: ~/vlsi_test

```
module and_gate(input a, input b,output y);
assign y = a&b;
endmodule

dell@DellGopika:~/vlsi_test$ nano and_gate_tb.v
dell@DellGopika:~/vlsi_test$ ls
and_gate.v  and_gate_tb.v
dell@DellGopika:~/vlsi_test$ iverilog -o and_gate.out and_gate.v and_gate_tb.v
and_gate_tb.v:9: syntax error
and_gate_tb.v:9: error: Malformed statement
dell@DellGopika:~/vlsi_test$ cat and_gate_tb.v
module and_gate_tb;
reg a;
reg b;
wire y;
and_gate uut(.a(a),.b(b),.y(y));
initial
begin
$monitor("A=%b B= %b y =%b",a,b,y);
a =0;b-0;
#10 a=0; b=1;
#10 a=1; b=0;
#10 a=1; b=1;
#10 $finish;
end
endmodule
dell@DellGopika:~/vlsi_test$ ^C
dell@DellGopika:~/vlsi_test$ nano and_gate_tb.v
dell@DellGopika:~/vlsi_test$ iverilog -o and_gate.out and_gate.v and_gate_tb.v
dell@DellGopika:~/vlsi_test$ _
```

dell@DellGopika: ~/vlsi_test

```
10 a=0; b=1;
10 a=1; b=0;
10 a=1; b=1;
10 $finish;
nd
ndmodule
ell@DellGopika:~/vlsi_test$ ^C
ell@DellGopika:~/vlsi_test$ nano and_gate_tb.v
ell@DellGopika:~/vlsi_test$ iverilog -o and_gate.out and_gate.v and_gate_tb.v
ell@DellGopika:~/vlsi_test$ ls
nd_gate.out  and_gate.v  and_gate_tb.v
ell@DellGopika:~/vlsi_test$ vvp and_gate.out
=0 B= 0 y =0
=0 B= 1 y =0
=1 B= 0 y =0
=1 B= 1 y =1
nd_gate_tb.v:13: $finish called at 40 (1s)
ell@DellGopika:~/vlsi_test$ nano and_gate_tb.v
ell@DellGopika:~/vlsi_test$
ell@DellGopika:~/vlsi_test$ iverilog -o and_gate.out and_gate.v and_gate_tb.v
ell@DellGopika:~/vlsi_test$ vvp and_gate.out
CD info: dumpfile and_gate.vcd opened for output.
=0 B= 0 y =0
=0 B= 1 y =0
=1 B= 0 y =0
=1 B= 1 y =1
nd_gate_tb.v:15: $finish called at 40 (1s)
ell@DellGopika:~/vlsi_test$ ls
nd_gate.out  and_gate.v  and_gate.vcd  and_gate_tb.v
ell@DellGopika:~/vlsi_test$
```

```
dell@DellGopika:~/vlsi_test$ gtkwave and_gate.vcd
[1] 7012
dell@DellGopika:~/vlsi_test$
GTKWave Analyzer v3.3.126 (w)1999-2026 BSI

[0] start time.
[40] end time.
WM Destroy
~
```

Figure 7.4: AND-gate simulation waveform viewed using GTKWave